

Ecole

백준 문제 풀이

2025.09.20

Python

수 정렬하기(버블 정렬)



핵심

문제[015] 수 정렬하기 1

시간 제한 2초 | 난이도 브론즈 I | 백준 온라인 저지 2750번

N개의 수가 주어졌을 때 이를 오름차순 정렬하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 수의 개수 N ($1 \leq N \leq 1,000$), 2번째 줄부터 N개의 줄에 숫자가 주어진다. 이 수는 절댓값이 1,000보다 작거나 같은 정수다. 수는 중복되지 않는다.

↑ 출력

1번째 줄부터 N개의 줄에 오름차순 정렬한 결과를 1줄에 1개씩 출력한다.

예제 입력 1

5
5
2
3
4
1

예제 출력 1

1
2
3
4
5



병합 정렬 구현

두 인접한 데이터의 크기를 비교해
정렬하는 방법

시간 복잡도는 $O(n^2)$ 으로 다른 정렬
알고리즘보다 속도가 느림

현재 값과 다음 값을 비교해 swap
을 통해 정렬

```
n = int(input())
result = []
for i in range(n):
    result.append(int(input()))

for i in range(n-1):
    for j in range(n-i-1):
        # 앞의 원소가 뒤의 원소보다 크면 swap
        if result[j] > result[j+1]:
            result[j], result[j+1] = result[j+1], result[j]

for i in range(n):
    print(result[i])
```


Java, JS

수 정렬하기(버블 정렬)



Java

```
public class P001 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));

        int n = Integer.parseInt(br.readLine());

        int[] numList = new int[n];
        for(int i = 0; i < n; i++) {
            numList[i] = Integer.parseInt(br.readLine());
        }
        for(int i = 0; i < n-1; i++) {
            for(int j = 0; j < n-1-i; j++) {
                if (numList[j] > numList[j+1]) {
                    int temp = numList[j];
                    numList[j] = numList[j+1];
                    numList[j+1] = temp;
                }
            }
        }

        for (int i : numList) {
            bw.write(str: i + "\n");
        }
        bw.flush();
        bw.close();
        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');

const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = Number(input.shift());

const result = input.map(Number);

for (let i = 0; i < n-1; i++) {
    for (let j=0; i < n-1-i; j++) {
        if (result[j] > result[j+1]) {
            // JS의 '배열 구조 분해 할당' 사용
            [result[j], result[j+1]] = [result[j+1], result[j]];
        }
    }
}

console.log(result.join('\n'));
```

Python

내림차순으로 자릿수 정렬하기

**문제[017] 내림차순으로 자릿수 정렬하기**

시간 제한 2초 | 난이도 실버 V | 백준 온라인 저지 1427번

배열을 정렬하는 것은 쉽다. 수가 주어지면 그 수의 각 자릿수를 내림차순으로 정렬하시오.

↓ 입력

1번째 줄에 정렬할 수 N이 주어진다. N은 1,000,000,000보다 작거나 같은 자연수다.

↑ 출력

1번째 줄에 자릿수를 내림차순 정렬한 수를 출력한다.

예제 입력 1

2143

예제 출력 1

4321



선택 정렬 구현

본 문제는 내림차순으로 정렬이므로
리스트에서 최댓값을 찾아 맨 앞으
로 스왑, 인덱스를 옮겨 그 뒤부터
최댓값을 찾아 스왑 하는 형식의
알고리즘을 이용하여 정렬

Python

```
n = input()
numList = list(map(int, n))
numList.sort(reverse=True)
for i in numList:
    print(i, end='')

# 선택 정렬
n = input()
numList = list(map(int, n))

for i in range(len(numList)):
    maxIndex = i
    for j in range(i + 1, len(numList)):
        if numList[j] > numList[maxIndex]:
            maxIndex = j

    numList[i], numList[maxIndex] = numList[maxIndex], numList[i]

for i in numList:
    print(i, end='')
```

Java, JS

내림차순으로 자릿수 정렬하기



Java

```
public class P002 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        String num = br.readLine();

        int[] numList = new int[num.length()];
        for(int i=0; i<num.length(); i++) {
            numList[i] = Integer.parseInt(num.substring(i, i+1));
        }

        for(int i=0; i<numList.length; i++) {
            int max = i;
            for (int j=i+1; j<numList.length; j++) {
                if(numList[j] > numList[max]) {
                    max = j;
                }
                if (numList[i] < numList[max]) {
                    int tmp = numList[i];
                    numList[i] = numList[max];
                    numList[max] = tmp;
                }
            }
        }

        for (int i : numList) {
            System.out.print(i);
        }
    }
}
```

JS

```
const fs = require('fs');

const input = fs.readFileSync('/dev/stdin').toString().trim();

const numList = input.split('').map(Number)

for (let i=0; i<numList.length; i++) {
    let maxIndex = i;
    for (let j=i+1; j<numList.length; j++) {
        if(numList[j] > numList[maxIndex]) {
            maxIndex = j;
        }

        [numList[i], numList[maxIndex]] = [numList[maxIndex], numList[i]];
    }

    console.log(numList.join(''));
```

Python

ATM 인출 시간 계산하기



문제[018] ATM 인출 시간 계산하기

시간 제한 1초 | 난이도 실버 III | 백준 온라인 저지 11399번

인하은행에는 ATM이 1대밖에 없다. 지금 이 ATM 앞에 N 명의 사람들이 줄을 서 있다. 사람은 1번에서 N 번까지 번호가 매겨져 있으며, i 번 사람이 돈을 인출하는 데 걸리는 시간은 P_i 분이다.

사람들이 줄을 서는 순서에 따라서 돈을 인출하는 데 필요한 시간의 합이 달라진다. 예를 들어 총 5명이 있고, $P_1 = 3, P_2 = 1, P_3 = 4, P_4 = 3, P_5 = 2$ 일 때를 생각해 보자. $[1, 2, 3, 4, 5]$ 순서로 줄을 선다면 1번 사람은 3분 만에 돈을 뺄 수 있다. 2번 사람은 1번 사람이 돈을 뺄 때까지 기다려야 하므로 $3 + 1 = 4$ 분이 걸린다. 3번 사람은 1번, 2번 사람이 돈을 뺄 때까지 기다려야 하므로 총 $3 + 1 + 4 = 8$ 분이 걸린다. 4번 사람은 $3 + 1 + 4 + 3 = 11$ 분, 5번 사람은 $3 + 1 + 4 + 3 + 2 = 13$ 분이 걸린다. 즉, 각 사람이 돈을 인출하는 데 필요한 시간의 합은 $3 + 4 + 8 + 11 + 13 = 39$ 분이다. $[2, 5, 1, 4, 3]$ 순서로 줄을 선다면 2번은 1분, 5번은 $1 + 2 = 3$ 분, 1번은 $1 + 2 + 3 = 6$ 분, 4번은 $1 + 2 + 3 + 3 = 9$ 분, 3번은 $1 + 2 + 3 + 3 + 4 = 13$ 분이 걸리므로 각 사람이 돈을 인출하는 데 필요한 시간의 합은 $1 + 3 + 6 + 9 + 13 = 32$ 분이다. 이 순서보다 모든 사람이 돈을 인출하는 데 필요한 시간이 짧을 수는 없다.

줄을 서 있는 사람의 수 N 과 각 사람이 돈을 인출하는 데 걸리는 시간 P_i 가 주어졌을 때 각 사람이 돈을 인출하는 데 필요한 시간의 합의 최솟값을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 사람의 수 N ($1 \leq N \leq 1,000$), 2번째 줄에 각 사람이 돈을 인출하는 데 걸리는 시간 P_i ($1 \leq P_i \leq 1,000$)가 주어진다.

↑ 출력

1번째 줄에 각 사람이 돈을 인출하는 데 필요한 시간의 합의 최솟값을 출력한다.

예제 입력 1

```
5 // 데이터 개수
3 1 4 3 2
```

예제 출력 1

```
32
```



파이썬 내장 .sort() 사용

```
n = int(input())
pList = list(map(int, input().split()))

pList.sort()

result = 0
for i in range(1, n+1):
    result += sum(pList[0:i])

print(result)
```

삽입 정렬 구현

```
n = int(input())
pList = list(map(int, input().split()))

result = 0
for i in range(1, len(pList)):
    for j in range(i, 0, -1):
        if pList[j] < pList[j-1]:
            pList[j], pList[j-1] = pList[j-1], pList[j]
        else:
            break

for i in range(1, n+1):
    result += sum(pList[0:i])

print(result)
```

Java, JS

ATM 인출 시간 계산하기



Java

```

public class P003 {
    new *
    public static void main(String[] args) throws IOException {
        new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int n = Integer.parseInt(br.readLine());

        StringTokenizer st = new StringTokenizer(br.readLine());
        int[] pList = new int[n];
        int[] sList = new int[n];
        for(int i = 0; i < n; i++) {
            pList[i] = Integer.parseInt(st.nextToken());
        }

        for(int i=1; i < n; i++) {
            int insertPoint = i;
            int insertValue = pList[i];
            for (int j=i-1; j>=0; j--) {
                if (pList[j] < pList[i]) {
                    insertPoint = j + 1;
                    break;
                }
                if (j == 0) {
                    insertPoint = 0;
                }
            }
            for(int j=i; j>insertPoint; j--) {
                pList[j] = pList[j-1];
            }
            pList[insertPoint] = insertValue;
        }
        sList[0] = pList[0];
        for (int i = 1; i < n; i++) {
            sList[i] = sList[i - 1] + pList[i];
        }
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += sList[i];
        }
        System.out.println(sum);
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = parseInt(input[0]);

// 두 번째 줄의 값들을 공백으로 나눈 뒤, 각 요소를 숫자로 변환하여 배열에 저장합니다.
const pList = input[1].split(' ').map(Number);

// 1. 배열을 오름차순으로 정렬합니다.
pList.sort((a, b) => a - b);

let result = 0; // 최종 결과를 저장할 변수
let prefixSum = 0; // 각 단계까지의 누적 합을 저장할 변수

// 2. 누적 합의 총합을 계산합니다.
for (let i = 0; i < n; i++) {
    prefixSum += pList[i]; // 현재까지의 시간을 누적 합에 더합니다.
    result += prefixSum; // 누적 합을 최종 결과에 더합니다.
}

// 최종 결과를 출력합니다.
console.log(result);

```

Python

K 번째 수 구하기

**문제[019] K번째 수 구하기**

시간 제한 2초 | 난이도 실버 V | 백준 온라인 저지 11004번

수 N 개($A_1, A_2, \dots, A_N$)가 주어진다. A 를 오름차순 정렬했을 때 앞에서부터 K 번째에 있는 수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 N ($1 \leq N \leq 5,000,000$)과 K ($1 \leq K \leq N$), 2번째 줄에 $A_1, A_2, \dots, A_N$ 이 주어진다 ($-109 \leq A_i \leq 109$).

↑ 출력

A 를 정렬했을 때 앞에서부터 K 번째에 있는 수를 출력한다.



파이썬 내장 .sort() 사용

```
import math
n, k = map(int, input().split())
numList = list(map(int, input().split()))
def solution(n, k, numList):
    numList.sort()
    return numList[k-1]

print(solution(n, k, numList))
```

퀵 정렬 구현

```
# 퀵정렬 사용
n, k = map(int, input().split())
numList = list(map(int, input().split()))
def solution(n, k, numList):
    quickSort(0, n-1, k-1)
    return numList[k-1]

def quickSort(start, end, k):
    global numList
    if (start < end):
        pivot = partition(start, end)
        if (pivot < k):
            quickSort(pivot + 1, end, k)
        elif (k < pivot):
            quickSort(start, pivot - 1, k)
        else:
            return
```

```
def partition(start, end):
    global numList

    if (start + 1 == end):
        if numList[start] > numList[end]:
            swap(start, end)
        return end

    pivot = (start + end) // 2
    swap(start, pivot)
    pivot = start
    i = start + 1
    j = end
    while(i <= j):
        # 피벗보다 큰 수가 나올 때 까지 i 증가
        while numList[pivot] > numList[i] and i < len(numList) - 1:
            i += 1

        # 피벗보다 작은 수가 나올 때 까지 j 감소
        while(numList[pivot] < numList[j] and j > 0):
            j -= 1
        if i <= j:
            swap(i, j)
            i += 1
            j -= 1

    tmp = numList[pivot]
    numList[start] = numList[j]
    numList[j] = tmp
    return j

def swap(a, b):
    global numList
    numList[a], numList[b] = numList[b], numList[a]

print(solution(n, k, numList))
```

Java, JS

K 번째 수 구하기



Java

```
public class P004 { new *
    static int[] numList; 18 usages

    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int k = Integer.parseInt(st.nextToken());

        numList = new int[n];
        st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) {
            numList[i] = Integer.parseInt(st.nextToken());
        }

        quickSort( start: 0, end: n - 1, k: k - 1);
        System.out.println(numList[k - 1]);
    }

    private static void quickSort(int start, int end, int k) { 3 usages new *
        if (start < end) {
            int pivot = partition(start, end);
            if (pivot < k) {
                quickSort( start: pivot + 1, end, k);
            } else if (k < pivot) {
                quickSort(start, end: pivot - 1, k);
            } else {
                return; // pivot == k 이면 탐색 종료
            }
        }
    }
}
```

```
private static int partition(int start, int end) { 1 usage new *
    // 데이터가 2개인 경우 특별 처리
    if (start + 1 == end) {
        if (numList[start] > numList[end]) {
            swap(start, end);
        }
        return end;
    }

    // 중간 값을 피벗으로 선택하고 맨 앞으로 이동
    int mid = (start + end) / 2;
    swap(start, mid);

    int pivotIndex = start; // 피벗의 '인덱스'
    int i = start + 1;
    int j = end;

    while (i <= j) {
        // 피벗보다 큰 수가 나올 때까지 i 증가
        // Python 코드의 'i < len(numList) - 1' 와 'j > 0' 경계 조건도 그대로 유지
        while (i < numList.length - 1 && numList[pivotIndex] > numList[i]) {
            i++;
        }

        // 피벗보다 작은 수가 나올 때까지 j 감소
        while (j > 0 && numList[pivotIndex] < numList[j]) {
            j--;
        }

        if (i <= j) {
            swap(i, j);
            i++;
            j--;
        }
    }

    // 피벗 값을 최종 위치(j)로 이동
    int tmp = numList[pivotIndex];
    numList[start] = numList[j];
    numList[j] = tmp;

    return j;
}
```

```
private static void swap(int a, int b) { 3 usages new *
    int tmp = numList[a];
    numList[a] = numList[b];
    numList[b] = tmp;
}
```



JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');
const [n, k] = input[0].split(' ').map(Number);
const numList = input[1].split(' ').map(Number);

function partition(start, end) {
    // 데이터가 2개인 경우 특별 처리
    if (start + 1 === end) {
        if (numList[start] > numList[end]) {
            swap(start, end);
        }
        return end;
    }

    // 중간 값을 피벗으로 선택하고 맨 앞으로 이동
    const mid = Math.floor((start + end) / 2);
    swap(start, mid);

    const pivotIndex = start; // 피벗의 '인덱스'
    let i = start + 1;
    let j = end;

    while (i <= j) {
        // 피벗보다 큰 수가 나올 때까지 i 증가
        while (i < numList.length - 1 && numList[pivotIndex] > numList[i]) {
            i++;
        }
        // 피벗보다 작은 수가 나올 때까지 j 감소
        while (j > 0 && numList[pivotIndex] < numList[j]) {
            j--;
        }

        if (i <= j) {
            swap(i, j);
            i++;
            j--;
        }
    }

    // 피벗 값을 최종 위치(j)로 이동
    const tmp = numList[pivotIndex];
    numList[start] = numList[j];
    numList[j] = tmp;

    return j;
}
```

```
function quickSort(start, end, k) {
    if (start < end) {
        const pivot = partition(start, end);
        if (pivot < k) {
            quickSort(pivot + 1, end, k);
        } else if (k < pivot) {
            quickSort(start, pivot - 1, k);
        }
        // pivot === k 이면 암시적으로 탐색 종료
    }
}

function swap(a, b) {
    [numList[a], numList[b]] = [numList[b], numList[a]];
}

quickSort(0, n - 1, k - 1);
console.log(numList[k - 1]);
```

Python

수 정렬하기(병합 정렬)

**문제[020] 수 정렬하기 2**

시간 제한 2초 | 난이도 실버 V | 백준 온라인 저지 2751번

N개의 수가 주어졌을 때 이를 오름차순 정렬하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 수의 개수 N ($1 \leq N \leq 1,000,000$), 2번째 줄부터 N개의 줄에 숫자가 주어진다. 이 수는 절댓값이 1,000,000보다 작거나 같은 정수다. 수는 중복되지 않는다.

↑ 출력

1번째 줄부터 N개의 줄에 오름차순 정렬한 결과를 1줄에 1개씩 출력한다.



파이썬 내장 .sort() 사용

```
n = int(input())
numList = []
for _ in range(n):
    numList.append(int(input()))

numList.sort()
for i in range(0, n):
    print(numList[i])
```

병합 정렬 구현

```
n = int(input())
numlist = [int(input()) for _ in range(n)]

def mergeSort(numList):
    if len(numList) < 2:
        return numList

    mid = len(numList) // 2
    low = mergeSort(numList[:mid])
    high = mergeSort(numList[mid:])

    l, h = 0, 0
    merge = []
    while l < len(low) and h < len(high):
        if low[l] < high[h]:
            merge.append(low[l])
            l += 1
        else:
            merge.append(high[h])
            h += 1

    merge += low[l:]
    merge += high[h:]
    return merge

result = mergeSort(numlist)
for i in range(0, n):
    print(result[i])
```

Java, JS

수 정렬하기(병합 정렬)



Java

```
public class P005 {  
    static int[] numList, tmp;  
    public static void main(String[] args) throws IOException {  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));  
  
        int n = Integer.parseInt(br.readLine());  
        numList = new int[n+1];  
        tmp = new int[n+1];  
        for (int i = 1; i <= n; i++) {  
            numList[i] = Integer.parseInt(br.readLine());  
        }  
        mergeSort(1, n);  
  
        for(int i = 1; i <= n; i++) {  
            bw.write(str: numList[i] + "\n");  
        }  
        bw.flush();  
        bw.close();  
    }  
}
```

```
private static void mergeSort(int start, int end) {  
    if (end - start < 1) return;  
    int mid = start + (end - start) / 2;  
    mergeSort(start, mid);  
    mergeSort(mid + 1, end);  
    for(int i = start; i <= end; i++) {  
        tmp[i] = numList[i];  
    }  
    int k = start;  
    int index1 = start;  
    int index2 = mid + 1;  
    while (index1 <= mid && index2 <= end) {  
        if (tmp[index1] > tmp[index2]) {  
            numList[k] = tmp[index2];  
            k++;  
            index2++;  
        } else {  
            numList[k] = tmp[index1];  
            k++;  
            index1++;  
        }  
    }  
    while (index1 <= mid) {  
        numList[k] = tmp[index1];  
        k++;  
        index1++;  
    }  
    while (index2 <= end) {  
        numList[k] = tmp[index2];  
        k++;  
        index2++;  
    }  
}
```



JS

수 정렬하기(병합 정렬) - 정답

JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = parseInt(input[0]);
// input[0]은 n이므로, input[1]부터 숫자 리스트입니다.
const numList = input.slice(1).map(Number);

function mergeSort(arr) {
    // 배열의 길이가 2보다 작은 경우 return
    if (arr.length < 2) {
        return arr;
    }

    // 1. 분할 (Divide)
    const mid = Math.floor(arr.length / 2);
    // 중간을 기준으로 왼쪽(low)과 오른쪽(high) 배열로 나눕니다.
    // 재귀 호출을 통해 계속 분할.
    const low = mergeSort(arr.slice(0, mid));
    const high = mergeSort(arr.slice(mid));

    // 2. 병합 (Merge)
    const merged = [];
    let l = 0, h = 0;

    // 왼쪽(low)과 오른쪽(high) 배열의 요소를 하나씩 비교하며 작은 값을 새 배열에 추가합니다.
    while (l < low.length && h < high.length) {
        if (low[l] < high[h]) {
            merged.push(low[l]);
            l++;
        } else {
            merged.push(high[h]);
            h++;
        }
    }

    // 비교 후 남은 요소들을 새 배열의 뒤에 그대로 붙여줍니다.
    // 전개 구문(...)을 사용하면 더 간결하게 합칠 수 있습니다.
    merged.push(...low.slice(l));
    merged.push(...high.slice(h));

    return merged;
}

const result = mergeSort(numList);
console.log(result.join('\n'));
```


Python

수 정렬하기(기수 정렬)

**문제[022] 수 정렬하기**

N개의 수가 주어졌을

↓ **입력**

1번째 줄에 수의 개수

10,000보다 작거나 같은 정수 N이 주어진다.

↑ **출력**

1번째 줄부터 N개의 줄에 오름차순 정렬한 결과를 1줄에 1개씩 출력한다.

예제 입력 1

11
215
15
344
372
294
100
8
145
24
198
831

예제 출력 1

8
15
24
100
145
198
215
294
344
372
831

1



파이썬 내장 .sort() 사용

```
#메모리 초과
n = int(input())
numList = [int(input()) for _ in range(n)]
numList.sort()
for num in numList:
    print(num)
```

기수 정렬 구현

```
# 기수 정렬(메모리 초과)
from collections import deque
n = int(input())
numList = [int(input()) for _ in range(n)]
# 10진수에서 각 자릿수는 0부터 9까지 이므로 버킷은 10개 사용
buckets = [deque() for _ in range(10)]

maxVal = max(numList)
queue = deque(numList)
digit = 1

while(maxVal >= digit):
    while queue:
        num = queue.popleft()
        buckets[(num//digit)%10].append(num)

    for bucket in buckets:
        while bucket:
            queue.append(bucket.popleft())

    digit *= 10

for num in queue:
    print(num)
```

계수 정렬 구현

```
# 계수 정렬(통과)
n = int(input())
# 메모리 문제 해결을 위해 수의 범위 만큼의 카운트 배열만 생성
numList = [0]*10001
# 입력을 받아서 리스트에 저장하는게 아닌 바로 카운트 배열에 반영
for _ in range(n):
    numList[int(input())] += 1

# 카운트 배열을 순회하며 정렬된 결과를 출력
for i in range(10001):
    if numList[i] != 0:
        for _ in range(numList[i]):
            print(i)
```

Java, JS

수 정렬하기(기수 정렬)



Java

```
public class P006 {  
    static int[] numList;  
    public static void main(String[] args) throws IOException {  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));  
        int n = Integer.parseInt(br.readLine());  
        numList = new int[n];  
        for (int i = 0; i < n; i++) {  
            numList[i] = Integer.parseInt(br.readLine());  
        }  
        br.close();  
        RadixSort(numList, 5);  
        for (int i = 0; i < n; i++) {  
            bw.write(str: numList[i] + "\n");  
        }  
        bw.flush();  
        bw.close();  
    }  
}
```

```
private static void RadixSort(int[] numList, int maxSize) {  
    int[] output = new int[numList.length];  
    int digit = 1;  
    int count = 0;  
    while(count != maxSize) {  
        int[] bucket = new int[10];  
        for (int j : numList) {  
            bucket[(j / digit) % 10]++;  
        }  
        for (int i = 1; i < 10; i++) {  
            bucket[i] += bucket[i - 1];  
        }  
        for (int i = numList.length - 1; i >= 0; i--) {  
            output[bucket[(numList[i] / digit % 10)] - 1] = numList[i];  
            bucket[(numList[i] / digit % 10)]--;  
        }  
        for (int i = 0; i < numList.length; i++) {  
            numList[i] = output[i];  
        }  
        digit = digit * 10;  
        count++;  
    }  
}
```



JS

```
// 기수정렬 초과로 인한 계수정렬 코드
const fs = require('fs');

const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const n = parseInt(input[0]);
const counts = new Array(10001).fill(0);
for (let i = 1; i <= n; i++) {
    const num = parseInt(input[i]);
    counts[num]++;
}

const result = [];
for (let i = 1; i < counts.length; i++) {
    // counts[i]에 저장된 횟수만큼 숫자 i를 결과 배열에 추가합니다.
    if (counts[i] > 0) {
        for (let j = 0; j < counts[i]; j++) {
            result.push(i);
        }
    }
}

console.log(result.join('\n'));
```

Python

연결 요소의 개수 구하기



Python

연결 요소의 개수 구하기 - 문제설명

핵심

문제[023] 연결 요소의 개수 구하기

시간 제한 3초 | 난이도 실버 V | 백준 온라인 저지 11724번

방향 없는 그래프가 주어졌을 때 연결 요소^{connected component}의 개수를 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 노드의 개수 N ($1 \leq N \leq 1,000$)과 에지의 개수 M ($0 \leq M \leq N \times (N-1)/2$), 2번째 줄부터 M 개의 줄에 에지의 양끝 점 u 와 v 가 주어진다 ($1 \leq u, v \leq N, u \neq v$). 같은 에지는 한 번만 주어진다.

↑ 출력

1번째 줄에 연결 요소의 개수를 출력한다.

예제 입력 1

```
6 5 // 노드 개수, 에지 개수
1 2
2 5
5 1
3 4
4 6
```

예제 출력 1

```
2
```

예제 입력 2

```
6 8
1 2
2 5
5 1
3 4
4 6
5 4
2 4
2 3
```

예제 출력 2

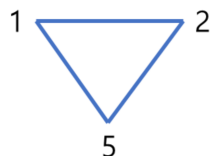
```
1
```




DFS

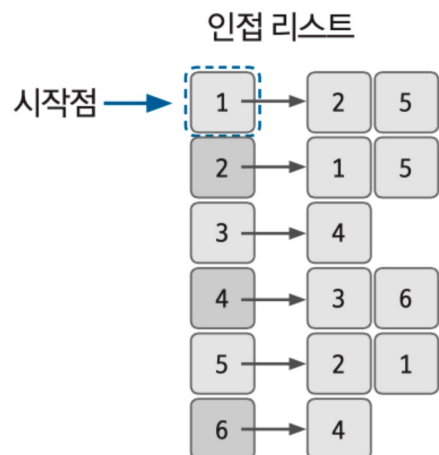
미로처럼 깊게 들어갔다가 방문했으면 다시 돌아오는 형식

특정 도시에서 다른 도시로 갈 수 있는지 없는지, 전자 회로에서 특정 단자와 단자가 서로 연결되어 있는지



예제 입력 1

```
6 5 // 노드 개수, 에지 개수
1 2
2 5
5 1
3 4
4 6
```



방문 배열

1	2	3	4	5	6
T	T	F	F	T	F

탐색 순서: 1 → 2 → 5

```
import sys
from collections import defaultdict

# 재귀 제한으로 인한 런타임 에러 방지(백준 기본 1000)
sys.setrecursionlimit(10**7)
input = sys.stdin.readline

n, m = map(int, input().split())

graph = [[] for _ in range(n+1)]
for _ in range(m):
    start, end = map(int, input().split())
    graph[start].append(end)
    graph[end].append(start)

visited = [False] * (n+1)

def dfs(v):
    visited[v] = True
    for i in graph[v]:
        if visited[i] == False:
            visited[i] = True
            dfs(i)

result = 0
for i in range(1, n+1):
    if visited[i] == False:
        result += 1
        dfs(i)

print(result)
```

Java, JS

연결 요소의 개수 구하기



Java

```

public class P007 { new *
    static ArrayList<Integer>[] graph; 5 usages
    static boolean[] visited; 4 usages
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        graph = new ArrayList[n + 1];
        visited = new boolean[n + 1];

        for (int i = 1; i <= n; i++) {
            graph[i] = new ArrayList<>();
        }

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int start = Integer.parseInt(st.nextToken());
            int end = Integer.parseInt(st.nextToken());

            graph[start].add(end);
            graph[end].add(start);
        }

        int componentCount = 0;
        for (int i = 1; i <= n; i++) {
            if (!visited[i]) {
                componentCount++;
                dfs(i);
            }
        }

        System.out.println(componentCount);
    }

    private static void dfs(int v) { 2 usages new *
        visited[v] = true;
        for (int neighbor : graph[v]) {
            if (!visited[neighbor]) {
                dfs(neighbor);
            }
        }
    }
}

```

JS

```

const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [n, m] = input[0].split(' ').map(Number);

const graph = Array.from({ length: n + 1 }, () => []);
const visited = new Array(n + 1).fill(false);

// 간선 정보를 입력받아 그래프를 구성합니다. (무방향 그래프)
for (let i = 1; i <= m; i++) {
    const [start, end] = input[i].split(' ').map(Number);
    // 양방향으로 간선을 추가합니다.
    graph[start].push(end);
    graph[end].push(start);
}

function dfs(v) {
    // 현재 정점을 방문 처리합니다.
    visited[v] = true;

    // 현재 정점과 연결된 모든 인접 정점들을 확인합니다.
    for (const neighbor of graph[v]) {
        // 인접 정점이 아직 방문하지 않은 상태라면
        if (!visited[neighbor]) {
            // 재귀적으로 DFS 호출
            dfs(neighbor);
        }
    }
}

let result = 0; // 연결 요소의 개수를 셀 변수

for (let i = 1; i <= n; i++) {
    // 아직 방문하지 않은 정점이라면
    if (!visited[i]) {
        result++; // 새로운 연결 요소이므로 개수를 증가시킵니다.
        dfs(i); // 해당 정점에서 DFS 탐색을 시작합니다.
    }
}

console.log(result);

```

Python

DFS와 BFS 프로그램



빈출

문제[026] DFS와 BFS 프로그램

시간 제한 2초 | 난이도 실버 II | 백준 온라인 저지 1260번

그래프를 DFS로 탐색한 결과와 BFS로 탐색한 결과를 출력하는 프로그램을 작성하시오. 단, 방문할 수 있는 노드가 여러 개일 경우에는 노드 번호가 작은 것을 먼저 방문하고 더 이상 방문할 수 있는 노드가 없을 때 종료한다. 노드 번호는 1에서 N까지다.

↓ 입력

1번째 줄에 노드의 개수 N ($1 \leq N \leq 1,000$), 에지의 개수 M ($1 \leq M \leq 10,000$), 탐색을 시작할 노드의 번호 V 가 주어진다. 그다음 M개의 줄에는 에지가 연결하는 두 노드의 번호가 주어진다. 어떤 두 노드 사이에 여러 개의 에지가 있을 수 있다. 입력으로 주어지는 에지는 양방향이다.

↑ 출력

1번째 줄에 DFS를 수행한 결과, 그다음 줄에 BFS를 수행한 결과를 출력한다. V부터 방문된 점을 순서대로 출력하면 된다.

예제 입력 1

```
4 5 1 // 노드 개수, 에지 개수, 시작점
1 2
1 3
1 4
2 4
3 4
```

예제 출력 1

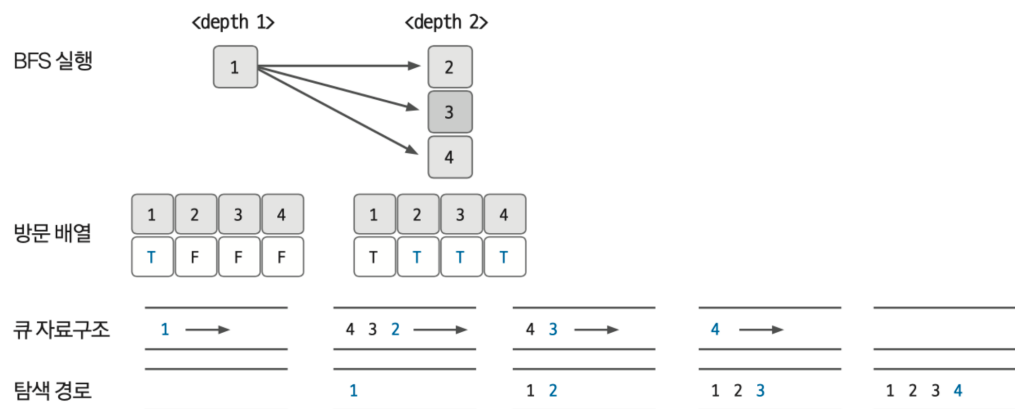
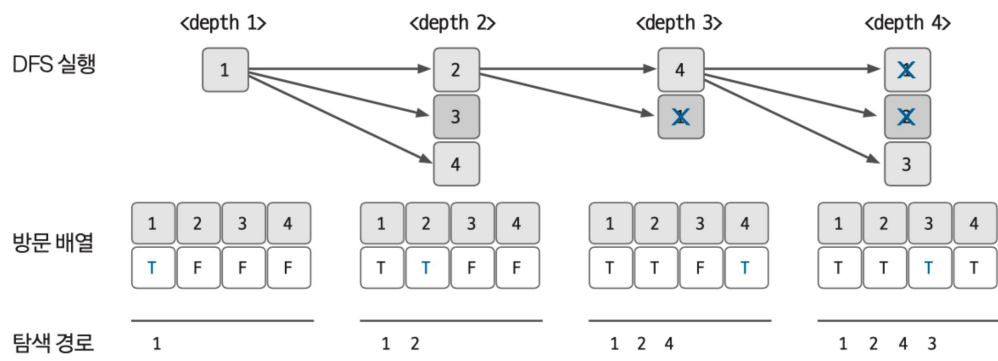
```
1 2 4 3
1 2 3 4
```

예제 입력 2

```
5 5 3
5 4
5 2
1 2
3 4
3 1
```

예제 출력 2

```
3 1 2 5 4
3 1 4 2 5
```



```
n, m, v = map(int, input().split())
graph = [[0]*(n+1) for _ in range(n+1)]
for i in range(m):
    a, b = map(int, input().split())
    graph[a][b] = graph[b][a] = 1

visitedDfs = [0]*(n+1)
visitedBfs = visitedDfs.copy()

def dfs(v):
    visitedDfs[v] = 1
    print(v, end = ' ')
    for i in range(1, n+1):
        if graph[v][i] == 1 and visitedDfs[i] == 0:
            dfs(i)

def bfs(v):
    queue = [v]
    visitedBfs[v] = 1
    while queue:
        v = queue.pop(0)
        print(v, end = ' ')
        for i in range(1, n+1):
            if (visitedBfs[i] == 0 and graph[v][i] == 1):
                queue.append(i)
                visitedBfs[i] = 1

dfs(v)
print()
bfs(v)
```

Java, JS

DFS와 BFS 프로그램



Java

```
public class P008 { new *
    static int n, m, v; 7 usages
    static int[][] graph; 5 usages
    static boolean[] visitedDfs; 3 usages
    static boolean[] visitedBfs; 4 usages
    static StringBuilder dfsResult = new StringBuilder(); 2 usages
    static StringBuilder bfsResult = new StringBuilder(); 2 usages

    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        v = Integer.parseInt(st.nextToken());

        graph = new int[n+1][n+1];
        visitedDfs = new boolean[n+1];
        visitedBfs = new boolean[n+1];

        for (int i = 0; i < m; i++) {
            st = new StringTokenizer(br.readLine());
            int a = Integer.parseInt(st.nextToken());
            int b = Integer.parseInt(st.nextToken());
            graph[a][b] = graph[b][a] = 1;
        }

        dfs(v);
        bfs(v);

        System.out.println(dfsResult);
        System.out.println(bfsResult);
    }
}
```

```
// DFS (깊이 우선 탐색) - 재귀
private static void dfs(int v) { 2 usages new *
    visitedDfs[v] = true;
    dfsResult.append(v).append(" ");

    for (int i = 1; i <= n; i++) {
        if (!visitedDfs[i] && graph[v][i] == 1) {
            dfs(i);
        }
    }
}

// BFS (너비 우선 탐색) - 큐
private static void bfs(int v) { 1 usage new *
    Queue<Integer> queue = new LinkedList<>();
    queue.add(v);
    visitedBfs[v] = true;

    while (!queue.isEmpty()) {
        int current = queue.poll();
        bfsResult.append(current).append(" ");

        for (int i = 1; i <= n; i++) {
            if (!visitedBfs[i] && graph[current][i] == 1) {
                queue.add(i);
                visitedBfs[i] = true;
            }
        }
    }
}
```




JS

```
const fs = require('fs');
const input = fs.readFileSync('/dev/stdin').toString().trim().split('\n');

const [n, m, v] = input[0].split(' ').map(Number);
const edges = input.slice(1);

// 그래프와 방문 배열 초기화
const graph = Array.from({ length: n + 1 }, () => new Array(n + 1).fill(0));
let visited = new Array(n + 1).fill(false);

// 인접 행렬 방식으로 그래프 구성
for (const edge of edges) {
  const [a, b] = edge.split(' ').map(Number);
  graph[a][b] = 1;
  graph[b][a] = 1;
}

const dfsResult = [];
const bfsResult = [];

// DFS (깊이 우선 탐색) - 재귀 방식
function dfs(current) {
  visited[current] = true;
  dfsResult.push(current);

  for (let i = 1; i <= n; i++) {
    if (graph[current][i] === 1 && !visited[i]) {
      dfs(i);
    }
  }
}
```

```
// BFS (너비 우선 탐색) - 큐 방식
function bfs(start) {
  const queue = [start];
  visited[start] = true;

  while (queue.length > 0) {
    const current = queue.shift(); // 큐의 맨 앞에서 요소를 꺼냄
    bfsResult.push(current);

    for (let i = 1; i <= n; i++) {
      if (graph[current][i] === 1 && !visited[i]) {
        queue.push(i); // 큐의 맨 뒤에 요소를 추가
        visited[i] = true;
      }
    }
  }
}

// --- 실행 ---
dfs(v);

// BFS를 위해 방문 배열 초기화
visited.fill(false);

bfs(v);

console.log(dfsResult.join(' '));
console.log(bfsResult.join(' '));
```

감사합니다!